# NLP HW3

## Section 1: Perplexities and Corpora

### 2.

Using add-($\lambda$) smoothing with ($\lambda$ = 0.01), a vocabulary threshold of 3, and the `switchboard-small` training corpus, the model's perplexity per word is:

| File | Perplexity per token |
|---|---|
| `sample1` | 230.80401 |
| `sample2` | 316.53492 |
| `sample3` | 313.02213 |

Because the starter code includes EOS tokens, this calculation does too.

### 3.

Training on the larger `switchboard` corpus gives us the following results for the model's perplexity per word:

| File | Small-corpus (\log_2 p) | Large-corpus (\log_2 p) | Small-corpus perplexity | Large-corpus perplexity |
|---|---|---|---|---|
| `sample1` | -11948.49828 | -9837.75693 | 230.80401 | 88.26097 |
| `sample2` | -7226.41215 | -6048.92099 | 316.53492 | 123.87856 |
| `sample3` | -7336.75695 | -6053.12185 | 313.02213 | 114.53825 |

The log-probabilities become less negative and the perplexities decrease substantially when the larger training corpus is used. This is probably because the larger training corpus gives us more observations of words, and in turn, trigram contexts, which allows the model to gain more reliable probability estimates and rely less heavily on smoothing.

## Section 3: Evaluating a Text Classifier

### 3(a)

Using add-one smoothing and the following prior:

$$p(\text{gen}) = 0.7$$

The classifier made 69 errors among 270 development files:

$$\text{error rate} = \frac{69}{270} \times 100 = 25.56\%$$

### 3(c)

Every development file is classified as spam when:

$$p(\text{gen}) < 2.51 \times 10^{-497}$$

This value is smaller than the smallest positive standard Python floating-point number.

Therefore, in practice, I had to use:

$$p(\text{gen}) = 0$$

### 3(d)

The development-set cross-entropies were:

| Lambda | Gen cross-entropy | Spam cross-entropy |
|---|---|---|
| 5 | 11.05263 | 11.07215 |
| 0.5 | 10.15485 | 10.26566 |
| 0.05 | 9.29458 | 9.44152 |
| 0.005 | **9.04616** | **9.09572** |
| 0.0005 | 9.49982 | 9.41952 |

The minimum gen cross-entropy was 9.04616 bits per token, and the minimum spam cross-entropy was 9.09572 bits per token. Both were achieved with:

$$\lambda = 0.005$$

### 3(e)

Using the same smoothing value for both models:

| Lambda | Combined cross-entropy |
|---|---|
| 5 | 11.06139 |
| 0.5 | 10.20457 |
| 0.05 | 9.36051 |
| 0.005 | **9.06840** |
| 0.0005 | 9.46379 |

Therefore:

$$\lambda^* = 0.005$$

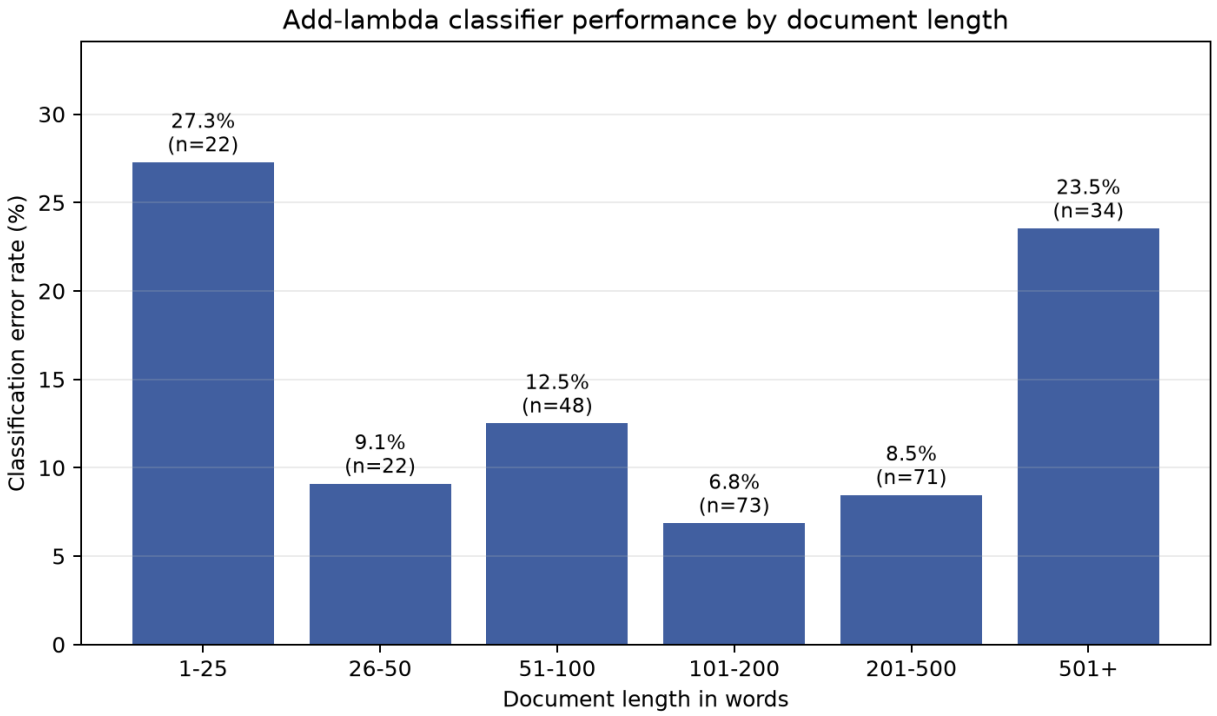
The minimum combined cross-entropy was:

$$9.06840 \text{ bits per token}$$

### 3(f)

I grouped the files into length ranges and measured the classification error rate in each range.

| Document length | Number of files | Error rate |
|---|---|---|
| 1–25 | 22 | 27.27% |
| 26–50 | 22 | 9.09% |
| 51–100 | 48 | 12.50% |
| 101–200 | 73 | 6.85% |
| 201–500 | 71 | 8.45% |
| 501+ | 34 | 23.53% |




Performance was best for medium-length files, especially those containing 101–200 words. Very short files performed poorly because they contained less evidence for classification. The longest files also had a relatively high error rate, suggesting that additional length doesn't always provide useful evidence for specific categories.

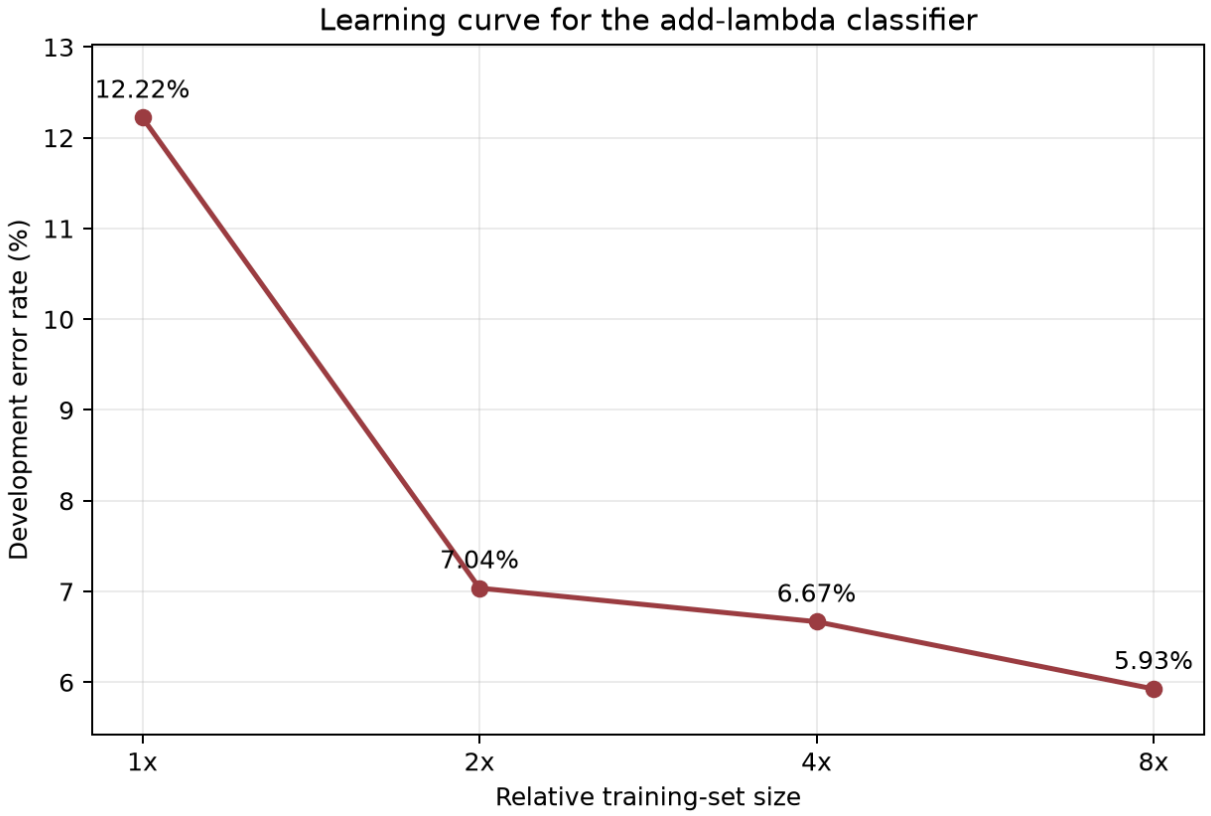
3(h)

Using:

$$\lambda^* = 0.005$$

The learning curve was:

| Relative training size | Development error rate |
|---:|---:|
| 1x | 12.22% |
| 2x | 7.04% |
| 4x | 6.67% |
| 8x | 5.93% |




Increasing the amount of training data consistently reduced the development error rate, although the improvements became smaller. Even with much more training data, I wouldn't expect the error rate to reach zero. Some genuine and spam emails look very similar, so a trigram model will probably continue to confuse a few of them.

## Section 4: Analysis

### 4(a)

The vocabulary size must include `OOV`. If there are 19,999 observed word types, then:

$$V = 19{,}999 + 1 = 20{,}000$$

If the uniform model incorrectly used 19,999, it would assign the following probability to each of the 20,000 possible tokens:

$$\hat{p}(z \mid xy) = \frac{1}{19{,}999}$$

The probabilities would then sum to more than one:

$$\sum_z \hat{p}(z \mid xy) = \frac{20{,}000}{19{,}999} > 1$$

Therefore, it would not be a valid probability distribution.

The add-lambda estimate would have the same problem, because its numerators would sum to:

$$c(xy) + 20{,}000\lambda$$

However, the incorrectly calculated denominator would be:

$$c(xy) + 19{,}999\lambda$$

The numerators would sum to more than the denominator, so the resulting probabilities would again sum to more than one.

### 4(b)

Setting lambda to zero removes all smoothing:

$$\hat{p}(z \mid xy) = \frac{c(xyz)}{c(xy)}$$

If a trigram was never observed, its probability would be zero:

$$c(xyz) = 0 \quad \Longrightarrow \quad \hat{p}(z \mid xy) = 0$$

A single unseen trigram would therefore make the probability of an entire test sequence zero and its cross-entropy infinite.

If the context was also never observed, the estimate would become undefined:

$$\hat{p}(z \mid xy) = \frac{0}{0}$$

Therefore, setting lambda to zero prevents the model from assigning sensible probabilities to unseen events.

### 4(c)

With add-lambda backoff smoothing:

$$\hat{p}(z \mid xy) = \frac{c(xyz) + \lambda V \hat{p}(z \mid y)}{c(xy) + \lambda V}$$

When the trigram count is zero:

$$c(xyz) = 0$$

the estimate becomes:

$$\hat{p}(z \mid xy) = \frac{\lambda V \hat{p}(z \mid y)}{c(xy) + \lambda V}$$

The same reasoning gives:

$$\hat{p}(z' \mid xy) = \frac{\lambda V \hat{p}(z' \mid y)}{c(xy) + \lambda V}$$

These probabilities are not *necessarily* equal, because the model uses the backed-off bigram probabilities to distinguish between the two unseen trigrams. They are equal only if:

$$\hat{p}(z \mid y) = \hat{p}(z' \mid y)$$

The bigram estimate recursively backs off to a unigram estimate, and the unigram estimate finally backs off to the uniform distribution:

$$\hat{p}(z) = \frac{c(z) + \lambda V \left(\frac{1}{V}\right)}{c() + \lambda V} = \frac{c(z) + \lambda}{c() + \lambda V}$$

If both trigram counts are one, their estimates become:

$$\hat{p}(z \mid xy) = \frac{1 + \lambda V \hat{p}(z \mid y)}{c(xy) + \lambda V}$$

and:

$$\hat{p}(z' \mid xy) = \frac{1 + \lambda V \hat{p}(z' \mid y)}{c(xy) + \lambda V}$$

They are still not necessarily equal because their backed-off probabilities may differ. However, each receives the same contribution of one from its observed trigram count.

### 4(d)

The backoff estimate can be viewed as a mixture of the observed trigram estimate and the backed-off estimate:

$$\hat{p}(z \mid xy) = \frac{c(xy)}{c(xy) + \lambda V} \frac{c(xyz)}{c(xy)} + \frac{\lambda V}{c(xy) + \lambda V} \hat{p}(z \mid y)$$

As lambda increases, more weight is placed on the backed-off estimate and less weight is placed on the observed trigram counts, which produces stronger smoothing and makes rare trigram counts less influential.

For unseen trigrams, increasing lambda moves the estimate closer to the backed-off bigram probability. Because the lower-order estimates are also smoothed recursively, very large values of lambda eventually move the entire model toward the uniform distribution:

$$\hat{p}(z \mid xy) \longrightarrow \frac{1}{V}$$

## Section 6: Sampling from Language Models

### 6(a)

I compared two models trained on the same email corpus and vocabulary, with the first model using ordinary add-one smoothing:

$$\lambda = 1,$$

and the second model used add-lambda smoothing with recursive backoff:

$$\lambda = 0.005,$$

both having a maximum sentence length of 20 tokens and random seed 465.

#### Add-one model samples

```
1. particular wrote realise Pro names These learning spent Help own
'll switch lovely brought Take income Smokers hanging Wed junior ...
2. women Rooms semantic software self-pity sending Syndicate engine
DIGITAL provided cleaners TR Although enable trees chemist arrived
Building heat RUNNER ...
3. million says ( half who assessment inbox chemistry free amusing
conclusion noon poisonous given NOTICE Speech Become Imagination
address removal ...
4. scheduled fantastic 12pm application ratio himself unfortunately
staff cater applications FASHION They necessary chatroom must text
broth digital Church object ...
5. SUBJECT: fee store moping white non-refundable quicker evaluation
NE between quicker land connector December discussed upon Video called
```

```
weeks --- ...
6. mortgage Lane School enter duvets station match
nlx1orzngls0fiznnzit tv adieu Once Hoping Hey fastest bottle consider
Believe EVER frozen bring ...
7. overview hope package lines broth hanging advance FREE clearly
electoral face' engine arrange under batch choices World left sentence
parted ...
8. reported point acquired both defeat 22Libraries safety Although
entry Time walls database these balance records can Christmas draft
During recently ...
9. ITS submit SUNDAY transfer prevent 'Stop Can gone restaurant ratio
opt-out purchases wines please guess direct mess wastelands valid
Company ...
10. SUBJECT: OOV schema appetising Porters wil Those opened strides
&ORG NO disgust Computer semantics transport overrun Fund college
close conflict ...
```

Backoff model samples

```
1. SUBJECT: &NAME Language &NAME &NAME &NAME , " &NAME &NAME &NAME
&NAME " &NAME &NAME &NAME . <END_QUOTE>
2. SUBJECT: Sorry I have now ... you 're all about &NAME &NAME , Thank
you for your birthday , we ...
3. SUBJECT: OOV old ladies ( &NUM &CHAR &NUM months FREE * OOV OOV and
OOV extract you attend ! Love ...
4. SUBJECT: OOV practicals -- 12pm , Tuesday 4th &NAME at our flat ,
&NAME &NAME wrote : <END_QUOTE>
5. SUBJECT: Your disreputable but do we hang out of the games I 'm now
OOV this afternoon to correct sentences ...
6. SUBJECT: Re : Repeat &NAME &NAME " wrote : <QUOTE> &NAME , &NAME ,
Once Hoping you ? OOV [ ...
7. SUBJECT: Re : ask &NAME , &NAME &NAME &NAME 's World . Email :
&EMAIL This . Speak to the ...
8. SUBJECT: &NAME &NAME wrote : <QUOTE> Dear &NAME , Do you need me "
what do n't plot , can ...
9. SUBJECT: OOV details - Computational Chemistry &NAME Group , I
apologise for that &NAME , the full . OOV skills ...
10. SUBJECT: Tutorial &NUM &NAME OOV , do a lot of the talk , sounds
fun they ca n't remember if ...
```

The add-one model ends up producing words that are mostly unrelated, and reaches the maximum length quite often. This is probably because add-one smoothing gives every possible continuation the same extra count, so unseen and unlikely words receive more probability than they should.

The backoff model produces more recognizable email structures, including `SUBJECT:`, `Re:`, names, quotations, and email addresses. When a trigram is rare or unseen, this model uses lower-order word frequencies instead of treating every possible continuation as equally probable.

The backoff samples are still grammatically inconsistent because a trigram model only considers the previous two words, but they do resemble the training emails much more closely than the add-one samples.

## Section 7: Implementing a log-linear model and training it with backpropagation

### 7(c)

I trained the basic log-linear models for 10 epochs using the `words-gs-only-10.txt` lexicon. For the larger experiments, I used mini-batches of 1024 trigrams to reduce the training time.

With

$$C = 1$$

the cross-entropies were:

- Genuine email: 11.12316 bits per token
- Spam email: 11.10278 bits per token
- Combined: 11.11402 bits per token

The complete regularization experiment with 10-dimensional embeddings was:

| Regularization | Gen cross-entropy | Spam cross-entropy | Combined cross-entropy |
|---|---|---|---|
| 0 | 11.12315 | 11.10277 | 11.11401 |
| 0.1 | 11.12315 | 11.10278 | 11.11401 |
| 0.5 | 11.12316 | 11.10278 | 11.11401 |
| 1 | 11.12316 | 11.10278 | 11.11402 |
| 5 | 11.12321 | 11.10283 | 11.11406 |

The lowest result technically came from no regularization, but the difference was extremely small. I selected

$$C^* = 0.1$$

because it was the best positive value and was effectively tied with the unregularized result.

Regularization did not matter much for the basic model, since it has a relatively small number of dense embedding features, and each feature receives evidence from many training tokens. There are therefore fewer rare, highly specific features that could freely overfit to the training data.

Holding

$$C^* = 0.1$$

fixed, I obtained:

| Embedding dimension | Gen cross-entropy | Spam cross-entropy | Combined cross-entropy |
|---:|---:|---:|---:|
| 10 | 11.12315 | 11.10278 | 11.11401 |
| 50 | 10.02006 | 9.94549 | 9.98660 |
| 200 | 8.62405 | 8.72176 | 8.66789 |

The best basic model therefore used the 200-dimensional `words-gs-only-200.txt` lexicon and

$$C^* = 0.1$$

Its combined cross-entropy of 8.66789 bits per token was better than the 9.06840 bits per token obtained by the best ordinary add-lambda model. The larger embeddings give us more information about the words, but the model still can't directly memorize unigram, bigram, or trigram frequencies individually.

Using a prior of

$$p(\text{gen}) = 0.7$$

the best basic model incorrectly classified 32 of the 270 development files, giving an error rate of 11.85%.

Searching over the prior found the best result near

$$p(\text{gen}) = 0.8044$$

This reduced the result to 31 errors, or 11.48%. Adjusting the prior therefore corrected one additional document, an improvement of about 0.37 percentage points.

I used the training data only to learn the model parameters, and the development data to select the regularization strength, embedding dimension, and prior probability. And, of course, I didn't use or inspect the test data since we use it for evaluations only once all the model and hyperparameter choices have been finalized.

I needed to use a fairly large genuine-email prior because the model was too willing to classify emails as spam. This effect became stronger for longer emails because every additional word contributes to the overall probability, but increasing the genuine-email prior helped balance out that bias.

At the same prior of 0.7, add-lambda backoff achieved a combined cross-entropy of 7.31417 bits per token and made 17 errors, giving an error rate of 6.30%. It therefore performed substantially better than the basic log-linear model on classification.

### 7(d)

I expanded the improved log-linear model with three kinds of features:

1. A learned weight for the log unigram count

$$\log(c(z) + 1),$$

2. A separate unigram indicator weight for every vocabulary word,
3. And a separate bigram indicator weight for every bigram that appeared at least three times in the training corpus.

The improved model still used the 200-dimensional embeddings. I used

$$C = 1$$

and a slightly larger learning rate for the unigram and bigram indicators because their feature values are much smaller than the dense embedding features.

The improved model achieved a combined development cross-entropy of 7.07739 bits per token. This is an improvement of 1.59051 bits per token over the basic log-linear model and 0.23678 bits per token over add-lambda backoff.

However, the classification error didn't improve in the same way. With

$$p(\text{gen}) = 0.7$$

the improved model made 37 errors, or 13.70%. Adjusting the prior to approximately

$$p(\text{gen}) = 0.9298$$

reduced this to 34 errors, or 12.59%.

The added frequency features therefore made the model substantially better at predicting the correct tokens, but that improvement did not translate into better spam classification. This shows that lower language-model cross-entropy and lower classification error are *related*, but are not exactly the same objective.

## 8. Speech Recognition

According to Bayes' Theorem, we should choose the candidate transcription with the highest posterior probability given the audio:

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmax}}\ p(\mathbf{w} \mid u)$$

Using Bayes' Theorem:

$$p(\mathbf{w} \mid u) = \frac{p(u \mid \mathbf{w})p(\mathbf{w})}{p(u)}$$

The denominator is identical for every candidate, so it can be ignored:

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmax}}\ p(u \mid \mathbf{w})p(\mathbf{w})$$

It is safer and simpler to perform this calculation in log space:

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmax}}\,[\log_2 p(u \mid \mathbf{w}) + \log_2 p(\mathbf{w})]$$

The first term is the acoustic score provided in the file, and the second term comes from the trigram language model:

$$\log_2 p(\mathbf{w}) = \sum_i \log_2 p(w_i \mid w_{i-2}, w_{i-1})$$

This calculation should include the probability of `EOS` and begin with the `BOS, BOS` context. Because the language model uses natural logarithms internally, its result should be divided by

$$\log 2$$

before it is added to the base-2 acoustic score.

In other words, we choose whichever transcription gets the best combined score from the audio and language models. We ignore the word-error rate when making this choice and only use it afterward to see how accurate our chosen transcription was!

## 9. Extra Credit: Language Modeling for Speech Recognition

### 9(a)

I implemented `speechrec.py` to choose among the nine candidate transcriptions in each utterance file.

For each candidate transcription, the program computes:

$$\log_2 p(u \mid \mathbf{w}) + \log_2 p(\mathbf{w})$$

The first term is the acoustic score supplied in the file. The second term is computed by the trigram language model:

$$\log_2 p(\mathbf{w}) = \sum_i \log_2 p(w_i \mid w_{i-2}, w_{i-1})$$

The language-model calculation starts with the `BOS, BOS` context and includes the final `EOS` probability. Since the language model uses natural logarithms internally, its result is converted to base 2 before being added to the acoustic score.

The program ignores the reference transcription and word-error-rate column when choosing a candidate. After making its choice, it uses the selected candidate's word-error rate to evaluate the result.

I also compute the overall word-error rate from the total number of errors rather than taking an unweighted average:

$$\text{WER} = \frac{\text{total word errors}}{\text{total words in the reference transcriptions}}$$

The implementation reproduces the handout's sample:

```
0.125 easy025
0.037 easy034
0.057 OVERALL
```

## 9. Extra Credit: Language Modeling for Speech Recognition

### 9(a)

I implemented `speechrec.py` to choose among the nine candidate transcriptions in each utterance file.

For each candidate transcription, the program computes:

$$\log_2 p(u \mid \mathbf{w}) + \log_2 p(\mathbf{w})$$

The first term is the acoustic score supplied in the file. The second term is computed by the trigram language model:

$$\log_2 p(\mathbf{w}) = \sum_i \log_2 p(w_i \mid w_{i-2}, w_{i-1})$$

The language-model calculation starts with the `BOS, BOS` context and includes the final `EOS` probability. Since the language model uses natural logarithms internally, its result is converted to base 2 before being added to the acoustic score.

The program ignores the reference transcription and word-error-rate column when choosing a candidate. After making its choice, it uses the selected candidate's word-error rate to evaluate the result.

I compute the overall word-error rate from the total number of errors instead of taking an unweighted average:

$$\text{WER} = \frac{\text{total word errors}}{\text{total words in the reference transcriptions}}$$

The implementation reproduces the handout's sample:

```
0.125 easy025
0.037 easy034
```

```
0.057 OVERALL
```

### 9(b)

I built the vocabulary from the full Switchboard training corpus with a threshold of 3, producing a vocabulary of 11,419 types. I compared uniform, add-lambda, and add-lambda-with-backoff models using only `dev/easy` and `dev/unrestricted`.

The best combined development result used add-lambda smoothing with recursive backoff and:

$$\lambda = 0.001$$

Its development results were:

| Development set | Word-error rate |
|---|---|
| `dev/easy` | 0.1497 |
| `dev/unrestricted` | 0.4640 |
| Combined | 0.2619 |

I selected this model before evaluating either test set. Its final test results were:

| Test set | Errors | Reference words | Word-error rate |
|---|---|---|---|
| `test/easy` | 194 | 1276 | 0.1520 |
| `test/unrestricted` | 350 | 902 | 0.3880 |

Backoff probably performed best because it could still use bigram and unigram evidence when a complete trigram was rare or unseen.